Lambert Solver and MGA Trajectory Optimizer: Implementation Notes and Benchmark Results

Technical Note v1.1

Stavros Kaloumenos
Department of Computer Science and Bioinformatics
University of Thessaly, Lamia, Greece
skaloumenos@uth.gr

9 April 2026

Abstract

This note documents two independent Python tools built from primary literature without external astrodynamics libraries. The first is a Lambert solver implementing Izzo's 2015 algorithm with third-order Householder iteration, validated against textbook solutions and NASA/JPL reference ephemeris data. The second is a differential evolution optimizer for multiple gravity assist (MGA) interplanetary trajectories, augmented with a multilayer perceptron surrogate for fitness pre-screening, and benchmarked against the ESA GTOP Cassini1 problem. Both tools constitute the Python foundation for a planned C++/OpenCL Solar Mission Planner described in a separate research proposal.

Contents

1 Motivation and Scope **3**

2 Lambert Solver **3**
2.1 Algorithm . . . 3
2.2 Ephemeris . . . 3
2.3 Validation Results . . . 4
2.4 Porkchop Plot Results: Earth → Mars 2026 . . . 4

3 MGA Trajectory Optimizer **4**
3.1 Differential Evolution . . . 4
3.2 MGA Fitness Function . . . 5
3.3 Neural Network Surrogate . . . 5

4 Results and Analysis **5**
4.1 Earth → Mars Direct (2D) . . . 5
4.2 Earth → Venus → Earth → Jupiter (4D) . . . 6
4.3 Cassini1 — EVVEJS (6D) . . . 6
4.3.1 Gap from the Reference Solution . . . 7
4.3.2 Surrogate Degradation on Cassini1 . . . 7
4.4 Cassini1-1DSM — EVVEJS (11D) . . . 8
4.4.1 Formulation Comparison . . . 8
4.5 Summary Comparison . . . 8

5 Relationship to the Solar Mission Planner **9**

6 Conclusions **9**

1 Motivation and Scope

Interplanetary trajectory design requires solving a sequence of tightly coupled subproblems: determining planetary positions at given epochs, solving Lambert's boundary-value problem to find transfer velocities between those positions, and searching a high-dimensional nonlinear space of departure dates and transfer times for trajectories that minimize propellant expenditure.

The tools described here address each layer independently, then compose them into a working MGA optimizer. The implementation constraint throughout was *no external astrodynamics libraries*: no astropy, no poliastro, no pykep, no pygmo. Every physical calculation is derived from first principles and cited to primary literature. This constraint serves two purposes: it ensures the author understands every line of the computation, and it produces code that can be directly ported to C++ without translation overhead from Python-specific library dependencies.

Both tools are publicly available at https://github.com/mendsergon.

2 Lambert Solver

2.1 Algorithm

The solver implements the algorithm from Izzo [1]. The Lambert boundary-value problem — given position vectors $\mathbf{r}_1$, $\mathbf{r}_2$ and a transfer time Δt, find the departure and arrival velocity vectors — is reformulated in terms of a single free parameter $x \in (-1, \infty)$, where $x < 1$ corresponds to elliptic transfers and $x > 1$ to hyperbolic.

The transfer geometry reduces to a single non-dimensional parameter:

$$\lambda = \pm\sqrt{1 - \frac{c}{s}} \tag{1}$$

where $c = |\mathbf{r}_2 - \mathbf{r}_1|$ is the chord length and $s = (r_1 + r_2 + c)/2$ is the semi-perimeter. The time of flight is non-dimensionalised as $T = \Delta t\sqrt{2\mu/s^3}$.

The equation $T(x) = T_{\text{target}}$ is solved via third-order Householder iteration using analytically computed derivatives:

$$T' = \frac{3Tx - 2 + 2\lambda^3 x/y}{1 - x^2} \tag{2}$$

$$T'' = \frac{3T + 5xT' + 2(1 - \lambda^2)\lambda^3/y^3}{1 - x^2} \tag{3}$$

$$T''' = \frac{7xT'' + 8T' - 6(1 - \lambda^2)\lambda^5 x/y^5}{1 - x^2} \tag{4}$$

This yields quartic convergence; in practice the solver terminates in 3–5 iterations. A Stumpff-function series expansion handles the parabolic singularity at $x = 1$ where the Battin time-of-flight expression is indeterminate.

2.2 Ephemeris

Heliocentric ecliptic (J2000) state vectors are computed from JPL approximate Keplerian elements with linear secular rates [2]. The procedure evaluates mean orbital elements at epoch, solves Kepler's equation via Newton–Raphson, converts to the perifocal frame, and rotates to J2000 ecliptic coordinates via a 3-1-3 Euler sequence (ω, I, Ω). The approximation is valid over 1800–2050 AD, with accuracy ± 1 arcmin for inner planets and ± 15 arcmin for outer planets — more than sufficient for porkchop plot generation, where Δv contours are dominated by transfer geometry rather than sub-arcminute ephemeris errors.

2.3 Validation Results

The solver passes all 13 validation tests. Key results are summarised in Table 1.

Table 1: Lambert solver validation results.

Test	Description	Result
BMW Ex. 5.2	Textbook Lambert (Earth gravity)	Energy/momentum to machine precision
Hohmann recovery	Circular coplanar Earth→Mars	$\Delta v_{\text{dep}} = 2.946$ km/s
Ephemeris check	Earth & Mars at 2026-01-01	Positions ±0.01 AU of JPL Horizons
Energy consistency	$\varepsilon_1 = \varepsilon_2$ at endpoints	Relative error $< 10^{-15}$
Angular momentum	$h_1 = h_2$ at endpoints	Relative error $< 10^{-15}$

2.4 Porkchop Plot Results: Earth → Mars 2026

Table 2 summarises the Earth–Mars 2026 porkchop plot.

Table 2: Earth→Mars 2026 porkchop plot results.

Property	Value
Optimal departure	2026-11-01
Optimal arrival	2027-09-07
Minimum total Δv	5.611 km/s
Departure Δv	3.041 km/s
Arrival Δv	2.570 km/s
Time of flight	310 days
Grid size	$81 \times 112 = 9{,}072$ solves
Compute time	2.8 s
Convergence rate	97.9%

The Hohmann transfer minimum of 5.596 km/s is recovered to within 0.3%. The small discrepancy arises from Mars' orbital eccentricity ($e \approx 0.093$) and inclination, which the Hohmann calculation ignores. The diagonal ridge visible in the contour plot is the 180° transfer singularity — a physically correct feature of the Lambert problem geometry.

3 MGA Trajectory Optimizer

3.1 Differential Evolution

The optimizer implements DE/rand/1/bin [3]. For each candidate $\mathbf{x}_i$ in the population, a trial vector is built as:

$$\mathbf{v} = \mathbf{x}_a + F \cdot (\mathbf{x}_b - \mathbf{x}_c) \quad \text{(mutation)} \tag{5}$$

$$\text{trial}_j = \begin{cases} v_j & \text{if rand()} < CR \\ x_{i,j} & \text{otherwise} \end{cases} \quad \text{(crossover)} \tag{6}$$

$$\mathbf{x}_i \leftarrow \text{trial} \quad \text{if } f(\text{trial}) < f(\mathbf{x}_i) \quad \text{(selection)} \tag{7}$$

where $F = 0.7$ and $CR = 0.9$. Out-of-bound values are reflected rather than clipped, preserving exploration pressure near boundaries.

3.2 MGA Fitness Function

For a planet sequence $[p_0, p_1, \ldots, p_{N-1}]$, the decision vector is $\mathbf{x} = [t_0, T_1, \ldots, T_{N-1}]$ where t_0 is the departure date in MJD2000 and T_k is the transfer time for leg k in days.

At each intermediate planet the flyby cost depends on whether the required turn angle α between $\mathbf{v}_{\infty,\text{in}}$ and $\mathbf{v}_{\infty,\text{out}}$ is geometrically feasible. The maximum achievable deflection at periapsis radius r_p is:

$$\sin\left(\frac{\delta_{\max}}{2}\right) = \frac{1}{1 + r_p v_\infty^2/\mu_{\text{planet}}} \tag{8}$$

The flyby cost is then:

$$\Delta v_{\text{flyby}} = \begin{cases} \big||\mathbf{v}_{\infty,\text{out}}| - |\mathbf{v}_{\infty,\text{in}}|\big| & \text{if } \alpha \le \delta_{\max} \\ \sqrt{v_1^2 + v_2^2 - 2v_1 v_2 \cos(\alpha - \delta_{\max})} & \text{if } \alpha > \delta_{\max} \end{cases} \tag{9}$$

The first case correctly reports zero cost for a geometrically feasible unpowered flyby where the magnitude mismatch can be accommodated. The second computes the minimum impulse at periapsis when the requested turn exceeds the physical limit.

3.3 Neural Network Surrogate

A multilayer perceptron (128, 128, 64 ReLU hidden units, Adam optimizer) is trained on accumulated fitness evaluations and used to pre-screen each generation's trial population. Inputs are normalized to $[0, 1]$ using problem bounds. Fitness values are clipped at 100 km/s before standardization to prevent extreme infeasibility penalties (10^6 km/s) from destroying training gradients. The surrogate is retrained every 10 generations on a rolling reservoir of up to 10,000 observations.

At each generation the full trial population is ranked by surrogate prediction, and only the top 30% trigger a real Lambert solve. The primary evaluation metric is Spearman rank correlation on held-out data rather than MAE — what matters for pre-screening is fitness *ordering*, not absolute accuracy.

4 Results and Analysis

4.1 Earth → Mars Direct (2D)

Table 3 reports 5-run statistics at a budget of 5,000 real evaluations per run.

Table 3: Earth→Mars direct: 5 runs × 5,000 evaluations.

Metric	DE	Surrogate-DE
Best Δv (km/s)	5.608	5.608
Mean Δv (km/s)	5.608	5.608
Std (km/s)	0.000	0.000
Avg evaluations to convergence	1,064	3,335
Avg wall time (s)	0.18	43.62
Reference (km/s)	5.610	

Best mission: departure 2026-10-30, arrival 2027-09-06, time of flight 311 days, total $\Delta v =$ 5.608 km/s. Agreement with the independent porkchop plot minimum (5.611 km/s) is within 0.05%, confirming that the DE optimizer and MGA fitness function are correctly coupled.

Both optimizers find the global minimum consistently across all 5 runs. The surrogate provides no benefit on this problem. DE converges in approximately 1,000 evaluations on a 2D landscape, so there is no evaluation budget to save — the surrogate warmup and retraining overhead make it 240× slower for identical solution quality. This is the expected behaviour: surrogate pre-screening is only beneficial when the real fitness function is expensive relative to surrogate inference cost, and when the budget is the binding constraint. Neither condition holds for a 2D problem.

4.2 Earth → Venus → Earth → Jupiter (4D)

Table 4 reports 10-run statistics at a budget of 50,000 real evaluations per run.

Table 4: EVEJ: 10 runs × 50,000 evaluations.

Metric	DE	Surrogate-DE
Best Δv (km/s)	11.743	11.743
Mean Δv (km/s)	12.085	12.852
Median Δv (km/s)	12.085	12.427
Worst Δv (km/s)	12.427	15.891
Std (km/s)	0.342	1.310
Avg wall time (s)	6.32	446.14

Best mission: departure 2026-08-07, Earth→Venus 130 days, Venus→Earth 316 days, Earth→Jupiter 1,058 days, total duration 1,504 days (4.12 years), total $\Delta v = 11.743$ km/s.

DE finds exactly two distinct basins across 10 runs: 11.743 km/s (5 runs) and 12.427 km/s (5 runs) with zero variance within each group. The std = 0.342 km/s arises entirely from the 50/50 split between basins, confirming the EVEJ landscape is bimodal at population 40. DE terminates at 9,000–11,000 evaluations out of a 50,000 budget — the same population constraint pattern observed on Cassini1, now confirmed at 4D.

The surrogate is slightly harmful at 4D: mean Δv increases by 6.4% and std rises from 0.342 to 1.310 km/s. Several surrogate-DE runs exhaust the full budget without triggering convergence, indicating the surrogate is delaying stagnation rather than preventing it. This is qualitatively different from Cassini1 where the surrogate caused active divergence — here it merely slows convergence without improving the final result.

4.3 Cassini1 — EVVEJS (6D)

Table 5 reports 5-run statistics at a budget of 30,000 real evaluations per run. The ESA GTOP Cassini1 problem encodes the Earth–Venus–Venus–Earth–Jupiter–Saturn sequence with decision vector $\mathbf{x} = [t_0, T_1, T_2, T_3, T_4, T_5]$.

Table 5: Cassini1 EVVEJS: 10 runs × 150,000 evaluations.

Metric	DE	Surrogate-DE
Best Δv (km/s)	9.419	9.419
Mean Δv (km/s)	9.711	11.669
Median Δv (km/s)	9.419	9.426
Worst Δv (km/s)	12.337	20.188
Std (km/s)	0.875	3.356
Avg wall time (s)	48.04	608.50
Reference MGA-1DSM (km/s)	4.930	

Best mission found:

Table 6: Best Cassini1 trajectory found by DE.

Leg	Transfer time	Arrival
Earth → Venus	180 days	1998-05-18
Venus → Venus	415 days	1999-07-06
Venus → Earth	53 days	1999-08-28
Earth → Jupiter	1,057 days	2002-07-20
Jupiter → Saturn	4,634 days	2015-03-28
Total	6,339 days	(17.35 years)

Departure date 1997-11-18 falls within 34 days of the real Cassini launch (1997-10-15), confirming that the problem geometry and ephemeris are correctly implemented.

4.3.1 Gap from the Reference Solution

The 9.419 km/s result is substantially above the 4.930 km/s ESA reference. Two independent factors account for this:

Formulation difference. This implementation uses unpowered MGA only — no deep-space impulses within transfer legs. The ESA best-known solution uses the MGA-1DSM transcription, which allows one deep-space maneuver per leg [4]. The search space here is strictly smaller, so the achievable minimum is higher by construction. This is a deliberate scope choice that keeps the fitness function physically transparent.

Population size is the binding constraint, not budget. At 150,000 evaluations with population 60, DE triggers its convergence tolerance and terminates at 39,000–65,000 evaluations across all 10 runs — well before the budget is exhausted. The non-zero standard deviation (std = 0.875 km/s, worst = 12.337 km/s) confirms the landscape is genuinely multimodal: different runs find different local basins and stagnate there. Increasing the budget further will not help because the convergence criterion fires before the budget is reached. The correct fix is a larger population (150–200) that maintains diversity long enough to escape local basins. This distinguishes the Cassini1 case from a pure budget problem — it is a diversity problem.

4.3.2 Surrogate Degradation on Cassini1

The surrogate-assisted DE performs significantly worse than plain DE on Cassini1: mean Δv increases by 20.2% and standard deviation rises from 0.875 to 3.356 km/s. This is a known failure mode of surrogate-assisted optimization on highly multimodal landscapes [5].

The MLP learns a smooth approximation of a jagged fitness surface. Pre-screening on this approximation systematically filters out candidates in unexplored basins that appear unpromising to the surrogate but lead to better optima. The result is reduced population diversity and premature stagnation. High variance across runs (9.419 to 20.188 km/s) confirms the surrogate is actively destabilizing the search.

Notably, increasing the budget from 30,000 to 150,000 evaluations improves the surrogate mean from 13.641 to 11.669 km/s — more observations help the MLP learn a better landscape approximation. However, it remains worse than plain DE at every budget tested, and the wall time penalty is severe (608.50 s vs 48.04 s per run). The surrogate is demonstrably useful only when the landscape is smooth enough for an MLP to learn useful fitness ordering. Cassini1 does not satisfy this condition at population 60.

The diagnostic is the held-out rank correlation during a run: when ρ degrades below a threshold the surrogate is learning noise rather than signal, and pre-screening should be disabled. This adaptive fallback is identified as the primary improvement for the next version.

4.4 Cassini1-1DSM — EVVEJS (11D)

Table 7 reports 10-run statistics for the MGA-1DSM formulation at 150,000 evaluations per run. The decision vector is $\mathbf{x} = [t_0, T_1, \eta_1, T_2, \eta_2, T_3, \eta_3, T_4, \eta_4, T_5, \eta_5]$ where $\eta_k \in (0, 1)$ is the fractional time along leg k where the deep-space maneuver occurs.

Table 7: Cassini1-1DSM EVVEJS: 10 runs × 150,000 evaluations.

Metric	DE
Best Δv (km/s)	9.419
Mean Δv (km/s)	9.419
Std (km/s)	0.000
Avg evaluations	109,823
Avg wall time (s)	475.3
Reference MGA-1DSM (km/s)	4.930

4.4.1 Formulation Comparison

Table 8 compares unpowered MGA and MGA-1DSM under identical experimental conditions.

Table 8: Unpowered MGA vs MGA-1DSM on Cassini1 EVVEJS.

Metric	Unpowered MGA (6D)	MGA-1DSM (11D)
Best Δv (km/s)	9.419	9.419
Mean Δv (km/s)	9.711	9.419
Std (km/s)	0.875	0.000
Population	60	110
Avg time per run (s)	48.04	475.3
Evals to convergence	40,000–65,000	100,000–120,000

The MGA-1DSM formulation finds the same best Δv as unpowered MGA. The DSM placements are non-trivial (η values ranging from 0.26 to 0.78 across legs) but the resulting DSM burns are near-zero — the optimizer has found the same trajectory family and placed maneuvers that contribute negligible cost. This confirms the population constraint finding extends to the 1DSM case: the extra 5 η variables expand the search space from 6D to 11D without helping escape the dominant local basin at 9.419 km/s. The correct fix remains population 150–200, not a different trajectory transcription.

The 1DSM formulation is approximately 10× slower per run (475 s vs 48 s) due to the Keplerian propagation sub-stepping required to locate DSM points on each transfer leg.

4.5 Summary Comparison

Table 9 contrasts the two problems across the key dimensions that determine whether surrogate assistance helps or hurts.

Surrogate damage scales systematically with dimensionality. As landscape complexity increases, the MLP's smooth approximation increasingly misrepresents the fitness surface, causing pre-screening to filter out promising candidates in unexplored basins. The time penalty per unit of

Table 9: Surrogate effectiveness scaling with problem dimensionality.

Property	Earth→Mars (2D)	EVEJ (4D)	Cassini1 (6D)
Dimensions	2	4	6
Population	20	40	60
Budget used by DE	∼1,000	9,000–11,000	40,000–65,000
Landscape	Smooth, unimodal	Bimodal	Highly multimodal
Surrogate Δv change	0.0%	+6.4%	+20.2%
Surrogate time penalty	240×	70×	13×
Conclusion	Unnecessary	Slightly harmful	Significantly harmful

quality degradation worsens at higher dimensions because surrogate retraining overhead grows while the benefit decreases. Across all three problems, population size rather than evaluation budget is the binding constraint: DE terminates well before exhausting its budget at every dimensionality tested.

5 Relationship to the Solar Mission Planner

These tools constitute the validated Python prototype of the computational core described in the Solar Mission Planner research proposal. The mapping is direct:

- The Lambert solver → transfer leg evaluation in the mission planning engine
- The analytical ephemeris → planetary state vector queries (to be replaced by full SPICE/ANISE ephemeris at higher fidelity)
- The DE optimizer → outer optimization loop (to be extended with self-adaptive variants and GPU-parallel population evaluation)
- The surrogate → fitness pre-screening layer (to be made adaptive based on held-out rank correlation monitoring)
- The MGA fitness function → candidate trajectory scoring (to be extended with MGA-1DSM, powered flybys, and aerobraking)

The planned C++ port will replace the Python implementation with OpenCL-accelerated parallel candidate evaluation, enabling population sizes of 500–1,000 at acceptable wall times. The Python codebase serves as the reference implementation against which the C++ port will be validated.

6 Conclusions

Two trajectory optimization tools were implemented from primary literature without external astrodynamics libraries. The Lambert solver achieves quartic convergence and recovers the Earth-Mars Hohmann minimum to within 0.3%. The MGA optimizer correctly solves the Earth-Mars direct problem to within 0.05% of the independent porkchop reference.

Benchmarking across three problems of increasing dimensionality reveals two systematic findings. First, population size rather than evaluation budget is the binding constraint at every dimensionality tested: DE terminates at ∼1,000 (2D), 9,000–11,000 (4D), and 40,000–65,000 (6D) evaluations regardless of available budget, leaving substantial budget unused at each scale. Second, surrogate damage scales with dimensionality: the MLP pre-screening is neutral at 2D

(identical quality, 240× slower), slightly harmful at 4D (+6.4% mean Δv, 70× slower), and significantly harmful at 6D (+20.2% mean Δv, 13× slower). Both findings directly motivate the v1.1 improvements: population 150–200 for Cassini1, and an adaptive surrogate fallback based on held-out rank correlation monitoring.

On ESA GTOP Cassini1 the best result is 9.419 km/s under both formulations. The MGA-1DSM extension (v1.1.0) adds a universal-variable Keplerian propagator and the split-leg DSM transcription, but finds the same trajectory family as unpowered MGA — confirming that the population constraint finding generalizes across formulations and dimensionalities. The gap from the 4.930 km/s reference is attributed to insufficient population diversity, not trajectory transcription.

Both tools are ready for C++ port and serve as the Python foundation for DAEDALUS.

References

[1] D. Izzo, "Revisiting Lambert's problem," *Celestial Mechanics and Dynamical Astronomy*, vol. 121, no. 1, pp. 1–15, 2015.

[2] E. M. Standish, "Approximate Positions of the Planets," JPL Solar System Dynamics, 1992 (updated 2022).

[3] R. Storn and K. Price, "Differential Evolution — A Simple and Efficient Heuristic for Global Optimization over Continuous Spaces," *Journal of Global Optimization*, vol. 11, no. 4, pp. 341–359, 1997.

[4] M. Vasile and P. De Pascale, "Preliminary Design of Multiple Gravity-Assist Trajectories," *Journal of Spacecraft and Rockets*, vol. 43, no. 4, pp. 794–805, 2006.

[5] Y. Jin, "Surrogate-assisted evolutionary computation: Recent advances and future challenges," *Swarm and Evolutionary Computation*, vol. 1, no. 2, pp. 61–70, 2011.

[6] M. Zuo et al., "A case learning-based differential evolution algorithm for global optimization of interplanetary trajectory design," *Applied Soft Computing*, vol. 94, p. 106451, 2020.